B. V. V. Sangha's
BASAVESHWAR ENGINEERING COLLEGE BAGALKOTE
DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING
Semester: 5th **Academic Year**: 2026-27

Mini Project Synopsis- BISA531P

1. Title of the Project

DebtLens: An Incident-Driven Technical Debt Intelligence and Remediation Platform

2. Introduction

Modern software teams operate under continuous delivery pressure. When a production defect or service failure occurs, the prevailing response is to restore service quickly and close the ticket. The corrective patch is recorded, but the underlying structural weakness that permitted the failure is rarely documented, quantified or scheduled for repair. Across successive releases these unrepaired weaknesses accumulate as technical debt: design and implementation compromises that remain in the codebase and silently increase the cost, duration and risk of every subsequent change.

Incident management, root cause analysis and technical debt management have each been studied extensively, but as independent concerns. Incident trackers record symptoms and assignees. Static analysis tools report code smells in isolation from the failures those smells actually caused. Neither preserves the causal link between the failure a team genuinely experienced and the structural weakness responsible for it, so remediation is argued for on intuition rather than evidence and consistently loses priority to feature delivery.

DebtLens treats that missing linkage as a first-class relationship. It ingests a project's own Git repository to derive objective evidence of structural weakness, captures each incident's root cause as a reusable node in a causal graph, prices the resulting technical debt as a ledger of principal and accruing interest, and recommends an optimal remediation set that fits the team's available effort budget. Evidence produces causality, causality produces economics, and economics produces a scheduling decision.

3. Problem Statement

When a software system fails, development teams typically restore service and close the incident without recording why the failure was possible. The structural weakness responsible remains in the codebase, undocumented, unquantified and unscheduled, until it produces the next failure. Three deficiencies follow from this practice.

- **Loss of causal traceability.** Incident records capture the symptom and the patch but not the reasoning connecting them, so the same underlying cause is rediscovered repeatedly by different developers across different modules.
- **Unquantified debt.** Teams cannot state what an item of technical debt is costing them. Without a cost figure, remediation is prioritised by seniority or intuition and loses every negotiation against new feature work.
- **Documentation dependence.** Existing tools demand exactly the discipline whose absence creates the problem. A system that only asks developers to record information will not be adopted by teams that do not already record it, so the system must derive evidence automatically and ask the developer only to confirm and enrich it.

Statement of the problem. To design and develop a system that automatically derives evidence of structural weakness from a project's own repository and incident history, preserves the causal chain from incident to root cause to technical debt, quantifies the accumulated cost of each debt item, and recommends an optimal remediation set within a bounded team effort budget.

4. Objectives

- **Platform foundation.** To build a multi-tenant web platform organised as Organisation, Project and Member, secured by JWT authentication and role-based access control for Admin, Technical Lead, Developer and Viewer.
- **Incident management.** To implement structured incident reporting with severity classification, affected module mapping and lifecycle tracking from Open through Investigating to Resolved.
- **Causal analysis.** To model guided 5 Whys analysis as a directed causal graph in which each cause is a reusable node, so that one root cause can link to many incidents and its systemic significance be measured by graph centrality.
- **Repository evidence.** To derive code churn, complexity hotspots, module dependency structure and self-admitted debt markers automatically from a linked Git repository, reducing dependence on manual documentation.
- **Incident intelligence.** To retrieve similar incidents, detect recurring issues, cluster root causes and identify regressions using a hybrid of TF-IDF lexical similarity and CPU-resident sentence-embedding semantic similarity.
- **Debt quantification.** To express every debt item as a principal, being the estimated remediation effort, and an accrued interest derived from incident cost, churn pressure and dependency blast radius, and to report its break-even status.
- **Remediation optimisation.** To select the subset of debt items maximising risk reduction within a bounded effort budget, formulated as a 0/1 knapsack problem over a topologically ordered dependency graph.
- **Knowledge and analytics.** To maintain a searchable record of incidents, resolutions and lessons learned, and to present debt balance, risk-ranked modules, aging profiles and what-if projections through dashboards and exportable PDF reports.
- **Deployment and evaluation.** To deploy the platform as a publicly accessible cloud application and to evaluate its analytical components quantitatively against a seeded corpus of realistic incident and repository data.

5. Scope of the Project

5.1 What the Project Covers

The project delivers a deployed web application supporting the software maintenance and evolution phase of the development life cycle, spanning incident intake, root cause analysis, technical debt registration, quantified prioritisation and remediation planning. Each project workspace is linked to one Git repository, from which the platform derives its supporting evidence.

5.2 Intended Users

- **Software developers.** Report incidents, conduct guided root cause analysis, register and resolve technical debt.
- **Technical leads and managers.** Review the debt ledger, set the sprint effort budget, approve or defer remediation.
- **Quality assurance engineers.** Monitor recurring defects and detect regressions against resolved incidents.
- **Project managers and owners.** Obtain quantified evidence to justify maintenance effort during release planning.
- **Academic users.** Students and instructors demonstrating technical debt management on real repositories.

5.3 Functional Modules

- **M1 Workspace.** Organisation, project and team management with role-scoped data isolation.
- **M2 Incidents.** Reporting, severity and module mapping, status workflow, search, filter and sort.
- **M3 Causal analysis.** Guided 5 Whys stored as a reusable causal graph, with resolution recording and centrality ranking.
- **M4 Debt registry.** Debt creation from incidents, linkage, impact and effort estimation, assignment, status and aging.
- **M5 Repository evidence.** Git and GitHub ingestion, churn and complexity hotspots, dependency graph, self-admitted debt detection.
- **M6 Intelligence.** Similar-incident retrieval, recurring-issue detection, root-cause clustering, regression detection.
- **M7 Ledger and optimiser.** Principal, interest, break-even, risk scoring, knapsack sprint planning, what-if simulation.
- **M8 Knowledge and reporting.** Searchable history, lessons learned, dashboards, trend charts, downloadable PDF reports.

5.4 Data Intake Paths

- **Seeded corpus.** Realistic incident, cause and debt records loadable in one action, so analytical features are meaningful from first use.
- **Repository ingestion.** Git and GitHub commits, issues and file-level history.
- **Bulk import.** CSV and JSON import for migrating an existing defect backlog.
- **Log ingestion.** Automatic incident raising from application error streams, planned as the final development phase.

5.5 Limitations

- **Language coverage.** Automated complexity and dependency extraction supports Python and JavaScript or TypeScript. Other languages are supported for incident and debt tracking but not for automated static evidence.
- **Effort estimation.** Remediation effort is supplied by developers. The system suggests a range from comparable historical fixes but does not infer effort automatically.
- **Repository scope.** One repository per project. Multi-repository and monorepo module mapping are outside the scope of this work.
- **Model validity.** The interest, risk and projection models are statistical heuristics validated against a seeded corpus, not calibrated against longitudinal industrial data. They are intended as decision support rather than authoritative financial measures.
- **Analysis boundary.** The platform analyses repository metadata and source text only. It does not build, execute or test the analysed code.

6. Literature Survey

[1] Potdar A. and Shihab E., "An Exploratory Study on Self-Admitted Technical Debt", IEEE International Conference on Software Maintenance and Evolution, 2014. Examining four large open-source systems, this foundational study established that developers frequently document their own compromises in source comments using markers such as TODO, FIXME and HACK, and that such debt is both prevalent and reliably detectable from comment text. It motivates the automated debt-candidate detection in our Repository Evidence Engine.

[2] Lenarduzzi V., Besker T., Taibi D. et al., "A Systematic Literature Review on Technical Debt Prioritization", Journal of Systems and Software, vol. 171, 2021. Reviewing 44 primary studies, the authors report that no consensus exists on how technical debt should be prioritised, that most strategies remain unvalidated in industry, and that tools rarely support decision making beyond simple listing. This establishes the gap our Payoff Optimiser addresses by expressing prioritisation as an explicit constrained optimisation.

[3] Ampatzoglou A., Mittas N., Tsintzira A. et al., "Exploring the Relation between Technical Debt Principal and Interest", Information and Software Technology, vol. 128, 2020. Using the Mantel test and linear mixed-effects modelling over ten Apache projects, this study confirms that artefacts with similar debt principal exhibit similar interest, identifying coupling and size as the dominant properties. It grounds our representation of debt as a principal-and-interest instrument.

[4] Tornhill A. and Borg M., "Code Red: The Business Impact of Code Quality", IEEE/ACM International Conference on Technical Debt, 2022. A quantitative study of 39 proprietary production codebases showing that low-quality code requires substantially longer development time and exhibits markedly higher defect density. It validates churn and complexity hotspot analysis as a proxy for the ongoing cost of unresolved debt, which our interest model adopts.

[5] Laney C. E., Barovic A. and Moin A., "Automated Duplicate Bug Report Detection in Large Open Bug Repositories", arXiv:2504.14797, 2025. This work evaluates six approaches to duplicate report detection, including topic modelling, clustering, deep learning and transformer-based summarisation, and proposes a threshold-based rather than top-k retrieval strategy. It informs our similar-incident retrieval, particularly the choice of a similarity threshold over fixed-rank retrieval.

Identified research gap. The reviewed literature treats incident analysis, self-admitted debt detection and technical debt prioritisation as separate problems. No widely available system maintains the causal linkage from an incident, through its root cause, to the technical debt that incident reveals, and then quantifies the accumulated interest of that debt to drive remediation scheduling. DebtLens is positioned at this intersection.

7. Proposed Methodology

7.1 System Architecture

A three-tier architecture is adopted. A React and TypeScript single-page application forms the presentation tier. A Python FastAPI service forms the application tier, exposing a REST API and hosting the analytics engine. A PostgreSQL database forms the data tier, storing incidents, causal nodes, debt items and repository evidence in a normalised relational schema in which the causal graph is held as an adjacency relation.

7.2 Core Processing Pipeline

The four analytical capabilities form a single value chain rather than four independent features.

- **Stage 1, Evidence.** The Repository Evidence Engine reads the linked repository, extracts commit history with PyDriller, computes per-file churn over a rolling window, derives cyclomatic complexity, builds the module dependency graph from import statements, and scans comments for self-admitted debt markers.
- **Stage 2, Causality.** An incident is passed through guided 5 Whys analysis. Each answer is normalised and stored as a cause node, and where a semantically equivalent node exists it is reused, causing the graph to converge over time. Degree and betweenness centrality then identify the most systemically significant weaknesses.
- **Stage 3, Economics.** Each debt item is priced. Its principal is the developer-supplied remediation effort. Its interest accrues from the incidents traced to it, weighted by severity and resolution effort, and amplified by the churn and blast radius supplied by Stage 1.
- **Stage 4, Decision.** The Payoff Optimiser consumes the priced ledger and returns the remediation set maximising risk reduction within the team's declared effort budget for the iteration.

7.3 Analytical Formulations

Incident similarity is a weighted hybrid of lexical and semantic matching, so the system stays functional without network access while benefiting from semantics when the embedding model is loaded.

- $sim(i, j) = \alpha \cdot \cos(tfidf_i, tfidf_j) + (1 - \alpha) \cdot \cos(emb_i, emb_j)$, where embeddings are 384-dimensional vectors from the all-MiniLM-L6-v2 sentence-transformer running on CPU.
- Root causes are clustered by DBSCAN over the embedding space using cosine distance, which needs no predetermined cluster count and isolates one-off incidents as noise.
- Incident i is a regression of resolved incident j when $sim(i, j) \geq \tau$, both affect the same module, and i occurs after the recorded resolution time of j.

The accrued interest of a debt item d is the historical cost it has already inflicted, amplified by structural exposure.

- $I(d) = \Sigma$ [severity_weight(k) · resolution_effort(k)] over all incidents k traced to d, plus $\lambda_1 \cdot$ churn(d) plus $\lambda_2 \cdot$ blast_radius(d), where blast_radius is the number of modules transitively dependent on the module of d, found by depth-first traversal of the dependency graph.
- A debt item is past break-even when I(d) exceeds its principal P(d): the team has already spent more coping with the debt than removing it would cost.

Remediation planning is a constrained optimisation over the priced ledger.

- Maximise $\Sigma R(d) \cdot x(d)$ subject to $\Sigma P(d) \cdot x(d) \leq B$ and $x(d) \in \{0, 1\}$, where R(d) is composite risk reduction, P(d) the effort principal and B the sprint budget. It is solved by dynamic programming over quantised effort units, with prerequisite relations enforced by topologically ordering the debt dependency graph before packing.

7.4 Development Methodology

An iterative and incremental process is adopted, each phase producing a demonstrable increment, with version control and continuous deployment maintained on GitHub throughout.

- **Phase 1, Foundation.** Schema design, authentication and access control, workspace and project management, incident CRUD, seeded corpus.
- **Phase 2, Causality and debt.** Causal graph, debt registry, incident-to-debt linkage, aging, knowledge base.
- **Phase 3, Intelligence and economics.** Repository ingestion, hotspot and self-admitted debt analysis, similarity and clustering, ledger, risk scoring, optimiser.
- **Phase 4, Presentation and release.** Dashboards, causal graph visualisation, what-if simulation, PDF reporting, log ingestion, evaluation and cloud deployment.

8. Software and Hardware Requirements

8.1 Software Requirements

- **Frontend.** React 18 with TypeScript, Vite, Tailwind CSS, Recharts for charts and Cytoscape.js for causal graph visualisation.
- **Backend.** Python 3.11, FastAPI, SQLAlchemy, Pydantic and Alembic, with JSON Web Tokens and bcrypt for security.
- **Database.** PostgreSQL 15.
- **Analytics.** scikit-learn for TF-IDF and DBSCAN, sentence-transformers with all-MiniLM-L6-v2, NetworkX for graph algorithms, NumPy and Pandas.
- **Repository analysis.** GitPython and PyDriller for commit mining, the GitHub REST API for issues, Radon for Python complexity and Madge for JavaScript dependencies.
- **Tooling.** WeasyPrint for PDF generation, with Git, GitHub, Visual Studio Code, Docker, Postman and pytest for development and testing.
- **Deployment.** Vercel for the frontend, Render for the API and Neon or Supabase for managed PostgreSQL, on Windows 10 or 11 or any modern Linux distribution.

8.2 Hardware Requirements

- **Development machine.** Intel Core i3 eighth generation or AMD Ryzen 3 and above, 8 GB RAM minimum with 16 GB recommended, and 20 GB of free disk space.
- **Graphics.** No dedicated GPU is required. The system is deliberately designed for CPU-only inference, the embedding model occupying approximately 90 MB.
- **Network.** A broadband connection for repository ingestion, package installation and deployment.
- **Deployment target.** A cloud container of 512 MB to 1 GB memory with a managed PostgreSQL instance, both within the free tiers of the selected providers.

9. Expected Outcome

- **A deployed platform.** A multi-tenant web application live at a public HTTPS address, supporting multiple organisations, projects and role-based users.
- **An end-to-end causal chain.** A reported incident traced through guided root cause analysis to a registered debt item, with that linkage preserved and queryable in both directions.
- **Technical debt expressed as a number.** For every debt item, its principal, accrued interest, break-even status and the specific incidents that generated its cost, converting debt from an opinion into an evidence-backed figure.
- **A generated remediation plan.** Given a sprint effort budget, the optimal debt subset by risk reduction, respecting prerequisite dependencies between items.
- **Evidence obtained without manual entry.** Ranked churn and complexity hotspots, a module dependency graph with computed blast radius, and self-admitted debt candidates mined directly from the repository.

- **Measured analytical performance.** Precision at k for similar-incident retrieval, cluster purity for root-cause grouping and accuracy for regression detection, all reported against the seeded evaluation corpus rather than asserted.
- **A continuable engineering artefact.** A documented, version-controlled repository with API documentation, database schema and deployment guide, suitable for extension into the major project.

Collectively these outcomes produce a system that does not merely record software failures but learns from them, converting a team's own incident history into quantified, prioritised and schedulable engineering action.

Project Associates:

Sl. No.	USN	Name	Signature
1.	2BA24IS171	Suraj H Deshpande	
2.	2BA24IS172	Sushma Hiremath	
3.	2BA24IS173	Sushma Krishnagouda Patil	
4.	2BA24IS174	Sushma Mathapati	

Project Guide:

Name	Designation	Signature
Prof. P. R. Muttannavar		